Lab Session

ELT Pipeline with dbt
GitHub Repository Scoring

Medallion Architecture: Bronze / Silver / Gold
Star Schema & Business Metrics

Module	Data Engineering Foundations
School	IMT Atlantique
Instructor	Rahma Gargouri
Format	In pairs
Stack	Python 3.9+, dbt-core 1.9+, dbt-duckdb, DuckDB
Deliverable	Fully functional dbt project (Git repo)

Bronze **Silver** **Gold**

March 2026

Contents

1 Introduction and Learning Objectives 3
1.1 Context . . . 3
1.2 Learning Objectives . . . 3
1.3 Recap — ELT vs ETL . . . 3
1.4 Overall Architecture . . . 3

2 Session Planning 3

3 Repositories to Analyze 4

4 Step 0 — Installation & Configuration 4
4.1 What is dbt? . . . 4
4.2 What is DuckDB? . . . 5
4.3 Installing Dependencies . . . 5
4.4 Initializing the dbt Project . . . 6
4.5 Configuring the DuckDB Profile . . . 6
4.6 Project Configuration . . . 6
4.7 Verification . . . 7

5 Step 1 — Data Ingestion: Bronze Layer 7
5.1 How Does the GitHub API Work? . . . 7
5.2 Provided Data . . . 8
5.3 Understanding the Ingestion Script . . . 9
5.4 Bronze Loading Script . . . 9
5.5 Source Declaration . . . 9

6 Step 2 — Silver Layer: Cleaning and Typing 10
6.1 Silver Model Schema . . . 10
6.2 Model `stg_repositories.sql` . . . 11
6.3 Model `stg_commits.sql` . . . 12
6.4 Model `stg_pull_requests.sql` . . . 12
6.5 Model `stg_issues` . . . 13
6.6 Silver Layer Verification . . . 13

7 Step 3 — Gold Layer: Star Schema 14
7.1 Recap — Star Schema . . . 14
7.2 Star Schema Design . . . 15
7.3 Dimension `dim_repository` . . . 15
7.4 Dimension `dim_date` . . . 16
7.5 Dimension `dim_contributor` . . . 17
7.6 Fact Table `fact_repo_activity` . . . 17

8 Step 4 — Scoring Table 18
8.1 Scoring Methodology . . . 18
8.2 Expected Interpretation . . . 19

9 Step 5 — Tests & Documentation 19
9.1 A) Generic Tests (YAML) . . . 19

9.2 B) Singular Tests (SQL) 20
9.3 Execution and Documentation 20
9.4 Full Pipeline (full refresh mode) 21

10 Step 6 — Incremental Materialization **21**
10.1 Why Go Incremental? 21
10.2 New Data for Testing 21
10.3 dbt Incremental Strategies 22
10.4 Testing Incremental 23

11 Step 7 — dbt Snapshots **24**
11.1 Why Snapshot? 25
11.2 Choosing the Table to Snapshot 25
11.3 Strategy: `check` vs `timestamp` 25
11.4 Running Snapshots 26

12 Expected Deliverables **26**

13 References and Resources **27**

1 Introduction and Learning Objectives

1.1 Context

You are a data engineer on a team that evaluates the **health and maturity of open source projects**. Your mission: build a complete ELT pipeline that, from raw GitHub API data, produces a **multi-criteria scoring** system to objectively compare 10 popular repositories.

This lab reproduces a real-world data engineering scenario: extraction, cleaning, dimensional modeling, and business metrics creation.

1.2 Learning Objectives

By the end of this lab, you will be able to:

1. **Design a complete ELT pipeline** using the Medallion architecture (Bronze / Silver / Gold).
2. **Use dbt** to organize, test, and document SQL transformations.
3. **Model a star schema**.
4. **Implement incremental strategies** to simulate a production pipeline.
5. **Use dbt snapshots** to capture change history (SCD Type 2).

1.3 Recap — ELT vs ETL

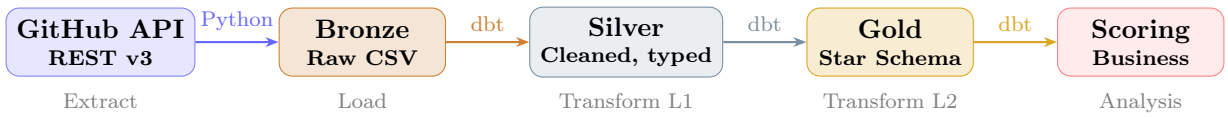
Why ELT?

In the traditional **ETL** approach, data is transformed *before* being loaded into the warehouse. This leads to fragile pipelines, difficult to debug, and the loss of raw data.
The **ELT** approach reverses the steps: first load raw data (Extract-Load), then transform *within* the warehouse (Transform). The advantages are significant:

- **Traceability**: raw data always remains accessible.
- **Reproducibility**: transformations can be replayed at any time.
- **Separation of concerns**: ingestion and transformation are decoupled.

dbt (data build tool) is the reference tool for orchestrating the "T" part of ELT.

1.4 Overall Architecture



2 Session Planning

The lab is divided into **8 progressive steps**. The pedagogical approach requires **first mastering full refresh mode** (steps 0–5), **then** introducing incremental and snapshots (steps 6–7).

Pedagogical Progression

Steps 0 through 5 build the complete pipeline in **full refresh mode**: load everything, transform everything, test everything. It is only **after** having a fully functional end-to-end pipeline that we introduce advanced concepts (incremental, snapshots) in steps 6 and 7. This progression reflects real-world practice: prototype in full mode first, then optimize for production.

3 Repositories to Analyze

The following 10 repositories cover diverse domains (web, ML, infrastructure, data) and offer an interesting spectrum in terms of activity, community size, and maturity.

#	Repository	Domain	Justification
1	`facebook/react`	Frontend web	Very active, large community
2	`tensorflow/tensorflow`	Machine Learning	Massive project, many contributors
3	`microsoft/vscode`	Editor / IDE	Sustained release pace
4	`torvalds/linux`	System / Kernel	Historic reference
5	`golang/go`	Language	Clear governance
6	`apache/spark`	Big Data	Data ecosystem
7	`langchain-ai/langchain`	LLM / AI	Recent project, strong growth
8	`pallets/flask`	Python Backend	Mature project, moderate pace
9	`docker/compose`	DevOps	Popular infra tool
10	`duckdb/duckdb`	Database	Relevant for this lab!

4 Step 0 — Installation & Configuration

Goal: operational working environment

By the end of this step, you will have an initialized dbt project, connected to DuckDB, with a clean directory structure and a passing `dbt debug`.

4.1 What is dbt?

dbt (data build tool) is an open-source tool that allows you to write data transformations in SQL, organize them as models, test them, and automatically generate documentation. dbt does **not** handle extraction or loading: it only manages the **T** in ELT.

Each dbt model is a `.sql` file containing a `SELECT` query. dbt takes care of creating the corresponding table or view in the database. Models can reference each other via the `{{ ref('model_name') }}` function, which automatically creates a dependency graph (lineage).

4.2 What is DuckDB?

DuckDB is an embedded analytical database, often described as the *"SQLite for analytics"*. Unlike traditional databases, DuckDB requires no server: it runs in a simple local file.

Why DuckDB is an excellent choice

Performance: DuckDB uses a vectorized columnar execution engine, making it extremely fast for analytical queries — often faster than PostgreSQL or MySQL for aggregations, even on a laptop.
Full SQL support: DuckDB supports advanced analytical SQL — window functions, recursive CTEs, QUALIFY, PIVOT, UNPIVOT, and even direct reading of CSV, Parquet, and JSON files without prior import.
Enterprise adoption: DuckDB is used in production by many companies for local data processing, analytical notebooks, and test pipelines. **MotherDuck**, the cloud service built on top of DuckDB, enables moving from local prototyping to cloud production without changing a single line of SQL — companies like Lego, Boeing, and Grafana Labs use it.
Ecosystem: DuckDB integrates natively with Python (pandas, Polars), R, and tools like dbt. It has become a preferred choice for *local data engineering* and *pipeline development*.
For this lab, DuckDB is ideal: zero configuration, free, fast, and SQL skills are directly transferable to Snowflake, BigQuery, or Microsoft Fabric.

4.3 Installing Dependencies

First create your project folder, then set up the Python environment inside it:

```
# Create the project folder and navigate into it
mkdir github_analytics
cd github_analytics

# Create a virtual environment, dbt is not supported in python 3.14
python3.12 -m venv .venv
source .venv/bin/activate     # Linux/Mac
# .venv\Scripts\activate      # Windows

# Install dbt with the DuckDB adapter
pip install dbt-core dbt-duckdb

# Verify versions
dbt --version
python -c "import duckdb; print(duckdb.__version__)"
```

```
(.venv) rahmagargouri@Rahmas-Air github_analytics % dbt --version
Core:
  - installed: 1.11.6
  - latest:    1.11.6 - Up to date!

Plugins:
  - duckdb: 1.10.1 - Up to date!


(.venv) rahmagargouri@Rahmas-Air github_analytics % python -c "import duckdb ; print ( duckdb . __version__ ) "
1.4.4
```

4.4 Initializing the dbt Project

What does `dbt init` do?

The `dbt init` command generates the **complete structure** of a dbt project: the `dbt_project.yml` configuration file, the `models/`, `tests/`, `snapshots/` folders, etc. It is an interactive command that asks for the project name and adapter type (here: `duckdb`). It is executed **only once** when starting the project.

```
# Initialize the dbt project in the current directory
dbt init
```

dbt will ask you a few questions (project name, database type). Answer `github_analytics` for the name and `duckdb` for the type.

Resulting Structure

At this point, your directory structure looks like this:

```
github_analytics/          # your working directory
    .venv/
    github_analytics/
        analyses/
        dbt_project.yml    # dbt configuration
        macros/
        models/
        seeds/
        snapshots/
        tests/
```

4.5 Configuring the DuckDB Profile

Create or edit the file `~/.dbt/profiles.yml` (On Windows: `C:\Users\<your_name>\.dbt\profiles.yml`

```yaml
github_analytics:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: "github_analytics.duckdb"
      threads: 4
```

Watch out for YAML indentation

YAML is very sensitive to indentation. Always use **spaces** (never tabs). Each indentation level = 2 spaces.

4.6 Project Configuration

Edit `dbt_project.yml` to define materialization per layer:

```yaml
name: "github_analytics"
version: "1.0.0"
```

```yaml
profile: "github_analytics"

model-paths: ["models"]
test-paths: ["tests"]

models:
  github_analytics:
    silver:
      +materialized: view
      +schema: silver
    gold:
      +materialized: table
      +schema: gold
```

View vs Table

- **view**: the query is recalculated on each read. Light on storage, always up to date.
- **table**: results are physically materialized. Faster to read, but must be recalculated by `dbt run`.

We use `view` for Silver (low cost) and `table` for Gold (read optimization).

Create the folder structure and delete the example/ folder:

```
mkdir -p models/bronze models/silver models/gold
rm -rf models/example
```

4.7 Verification

```
dbt debug
```

Checkpoint Step 0

You should see `All checks passed!` in the `dbt debug` output. If not, check your `profiles.yml` file (YAML indentation, DuckDB file path).

5 Step 1 — Data Ingestion: Bronze Layer

Bronze Layer — Raw Data

The Bronze layer stores data **as-is** from the source. No transformation, no cleaning. It is the faithful archive of what the API returned. If there is any doubt about downstream data, you can always go back to Bronze.

5.1 How Does the GitHub API Work?

The GitHub REST API (v3) exposes JSON endpoints to access repository data. Each endpoint returns a list of JSON objects. Without authentication, the limit is **60 requests per hour**.

The endpoints used in this lab are:

- `GET /repos/{owner}/{repo}` — **Repository metadata.** Returns a single JSON object with the *current state* of the repository: name, description, language, license, and live counters (`stargazers_count`, `forks_count`, `open_issues_count`, `watchers_count`). Unlike the other endpoints, this one does **not** return historical data — only a snapshot at the time of the call. If the star count changes tomorrow, yesterday's value is lost.

- `GET /repos/{owner}/{repo}/commits` — **Commit history.** Returns a paginated list of commits with their SHA, author login, author date, committer date, and commit message. This is historical data: each commit is an immutable event with a fixed timestamp.

- `GET /repos/{owner}/{repo}/pulls?state=all` — **Pull requests.** Returns all PRs (open, closed, merged) with PR-specific fields: `merged_at`, `draft`, `review_comments`, `additions`, `deletions`. Each PR has a lifecycle (`created_at` → `closed_at` / `merged_at`) that we can analyze.

- `GET /repos/{owner}/{repo}/issues?state=all` — **Issues.** Returns all issues (open and closed) with their title, state, labels, creation and close dates. **Important:** this endpoint also returns pull requests, because GitHub treats PRs as a special type of issue. We will filter them out in the Silver layer.

Why 4 endpoints and not 3?

GitHub's `/issues` endpoint also returns pull requests (because a PR is technically a special issue in GitHub). However, the `/issues` endpoint does **not** contain PR-specific fields that are essential for analysis: `merged_at` (merge date), `draft` (draft status), `review_comments` (code review effort). That is why we use both endpoints, and we will filter PRs from the `/issues` endpoint in the Silver layer.

5.2 Provided Data

Pre-extracted Data

The dataset was extracted on **December 31, 2025** and covers **3 months of history** (October, November, and December 2025). This means:

- `raw_repositories`$_history.csvcontainsasnapshotofeachrepository'smetadata$`as of`$December31$
- `raw_commits`$_history.csv$,

To avoid wasting time during the session on GitHub API rate limits, **the CSV files have been pre-extracted by the instructor**. Ask the instructor and them place them in a `data/raw/` folder **at the root of your project** (outside the dbt folder):

```
github_analytics/        # project root
  data/
    raw/
      raw_repositories_history.csv
      raw_commits_history.csv
      raw_pull_requests_history.csv
      raw_issues_history.csv
  github_analytics/      # dbt project
    models/
```

```
    tests/
    ...
```

5.3 Understanding the Ingestion Script

The extraction script `scripts/extract_github_history.py` handles the **Extract** phase of the ELT pipeline.

It connects to the GitHub REST API, queries 4 endpoints for each of the 10 repositories, and writes the results as CSV files in `data/raw/`. The script includes a `GitHubClient` class that manages API pagination and automatically pauses when approaching the rate limit.

For the three event-based endpoints (commits, PRs, issues), the script filters results to keep only activity within the history window (October–December 2025).

For the repository metadata endpoint, which only returns a real-time snapshot, the script simulates realistic values as of December 31, 2025 by back-projecting current metrics and walking them forward with random daily deltas.

A GitHub token is strongly recommended to raise the rate limit from 60 to 5,000 requests per hour.

5.4 Bronze Loading Script

Create the file `scripts/load_bronze.py`. This script loads the CSVs into DuckDB as Bronze tables (Again, ask the instructor for the script).

The duckdb database has to be created in the same level of dbt_project.yml.

5.5 Source Declaration

Create the file `models/bronze/_sources.yml`. This file declares the Bronze tables as dbt **sources**, allowing them to be properly referenced in Silver models and to apply tests.

```yaml
version: 2

sources:
  - name: bronze
    description: "Raw GitHub data"
    schema: main
    tables:
      - name: raw_repositories
        description: "Repository metadata"
        columns:
          - name: full_name
            description: "Unique identifier owner/repo"
            tests:
              - unique
              - not_null

      - name: raw_commits
        description: "Commit history"
        columns:
```

```yaml
          - name: sha
            tests:
              - not_null

      - name: raw_pull_requests
        description: "Pull requests (all states)"
        columns:
          - name: pr_number
            tests:
              - not_null

      - name: raw_issues
        description: "Issues (all states)"
        columns:
          - name: issue_number
            tests:
              - not_null
```

Checkpoint Step 1

Run the loading script and verify that all 4 tables are in DuckDB:

```bash
cd github_analytics     # dbt project folder
python ../scripts/load_bronze.py

# Verify
dbt show --inline "select count(*) as nb from bronze.raw_repositories
   "
dbt show --inline "select count(*) as nb from bronze.raw_commits"
dbt show --inline "select count(*) as nb from bronze.
   raw_pull_requests"
dbt show --inline "select count(*) as nb from bronze.raw_issues"
```

6 Step 2 — Silver Layer: Cleaning and Typing

Silver Layer — Cleaned and Typed Data

The Silver layer applies the first transformations: **type casting**, null value cleaning, normalization, and creation of simple derived columns. A Silver table generally corresponds to **one cleaned Bronze source**. The `stg_` (staging) prefix is the standard dbt convention.

6.1 Silver Model Schema

You will create **4 Silver models**, one per Bronze source:

Silver Model	Bronze Source	Key Transformations
stg_repositories	raw_repositories	Date typing, booleans, age calculation
stg_commits	raw_commits	Date typing, hour/day extraction
stg_pull_requests	raw_pull_requests	Typing, merge duration calculation
stg_issues	raw_issues	Typing, true issue filtering, duration

The {{ source() }} vs {{ ref() }} functions

- {{ source('bronze', 'raw_repositories') }} — References an external source (declared in _sources.yml).
- {{ ref('stg_repositories') }} — References another dbt model. dbt automatically manages execution order.

6.2 Model stg_repositories.sql

Your turn:

Objective: Clean and type repository metadata.
Expected transformations:

- Rename full_name to repo_id (business key).
- Cast dates (created_at, updated_at, pushed_at) to TIMESTAMP.
- Cast numeric metrics (stargazers_count, forks_count, etc.) to INTEGER.
- Replace null values: description → 'No description', language → 'Unknown'.
- Compute a derived column repo_age_days (difference between today and created_at).
- Exclude archived repositories (archived = true).

```
-- models/silver/stg_repositories.sql
{{ config(
    materialized='view'
) }}

with source as (
    select * from {{ source('bronze', 'raw_repositories') }}
),

cleaned as (

        -- TODO
)

select * from cleaned
```

6.3 Model stg_commits.sql

Your turn:

Objective: Clean commits and extract temporal information.
Expected transformations:

- Rename `sha` to `commit_sha`, `repo_full_name` to `repo_id`.
- Handle null `author_login` with `COALESCE`.
- Cast `author_date` and `committer_date` to `TIMESTAMP`.
- Extract `day_of_week` and `hour_of_day` from `author_date` (useful for analyzing development patterns).
- Truncate message to 200 characters.
- Filter rows where `sha IS NULL`.

```
-- models/silver/stg_commits.sql
{{ config(
    materialized='view'
) }}

with source as (
    select * from {{ source('bronze', 'raw_commits') }}
),

cleaned as (
    select
        -- TODO
    from source
    -- TODO
)

select * from cleaned
```

6.4 Model stg_pull_requests.sql

Your turn:

Objective: Clean PRs and compute merge duration.
Expected transformations:

- Rename and cast fields (dates to `TIMESTAMP`, `pr_number` to `INTEGER`).
- Create a boolean `is_merged` = `merged_at IS NOT NULL`.
- Create a boolean `is_draft` from the `draft` field.
- Compute `time_to_close_hours`: difference in hours between `created_at` and `merged_at` (if merged) or `closed_at` (if closed), otherwise `NULL`.

```
{{ config(
    materialized='view'
) }}

with source as (
    select * from {{ source('bronze', 'raw_pull_requests') }}
),
```

```
cleaned as (
    select
        -- TODO
    from source
    where pr_number is not null
)

select * from cleaned
```

6.5 Model stg_issues

Your turn:

Objective: Clean issues and filter out false issues (PRs).
Expected transformations:

- Rename and cast fields (same logic as other models).
- Cast `is_pull_request` to `BOOLEAN`.
- Compute `time_to_close_hours` (same logic as for PRs).
- **Filter out** pull requests: only keep rows where `is_pull_request = false`.

```
{{ config(
    materialized='view'
) }}

with source as (
    select * from {{ source('bronze', 'raw_issues') }}
),

cleaned as (
    select
        -- TODO

    from source
    where issue_number is not null
)

-- IMPORTANT: keep only REAL issues
select * from cleaned
-- TODO
```

Why filter `is_pull_request`?

GitHub's `/issues` endpoint also returns pull requests (because a PR is technically a special issue in GitHub). Without this filter, you would count PRs twice in your analyses. We analyze them separately via `stg_pull_requests` which contains PR-specific fields (`merged_at`, `review_comments`).

6.6 Silver Layer Verification

```
# Compile and run Silver models
dbt run --select silver
```

```
# Verify results
dbt show --inline "
  select repo_id, stars_count, repo_age_days
  from {{ ref('stg_repositories') }}
"
dbt show --inline "
  select repo_id, count(*) as nb_commits
  from {{ ref('stg_commits') }}
  group by repo_id
"
```

Checkpoint Step 2

All 4 Silver models must compile without errors. Verify that:

- Types are correct (dates as `TIMESTAMP`, integers as `INTEGER`).
- Derived columns have consistent values (`repo_age_days` > 0, `time_to_close_hours` >= 0).
- Issue filtering correctly excludes PRs.
- The row count in `stg_issues` is **lower** than in `raw_issues` (proof that the filter works).

7 Step 3 — Gold Layer: Star Schema

Gold Layer — Dimensional Model

The Gold layer organizes data into a **star schema** optimized for business analysis. It contains **dimension tables** (which describe business entities) and **fact tables** (which contain aggregated quantitative measures).

7.1 Recap — Star Schema

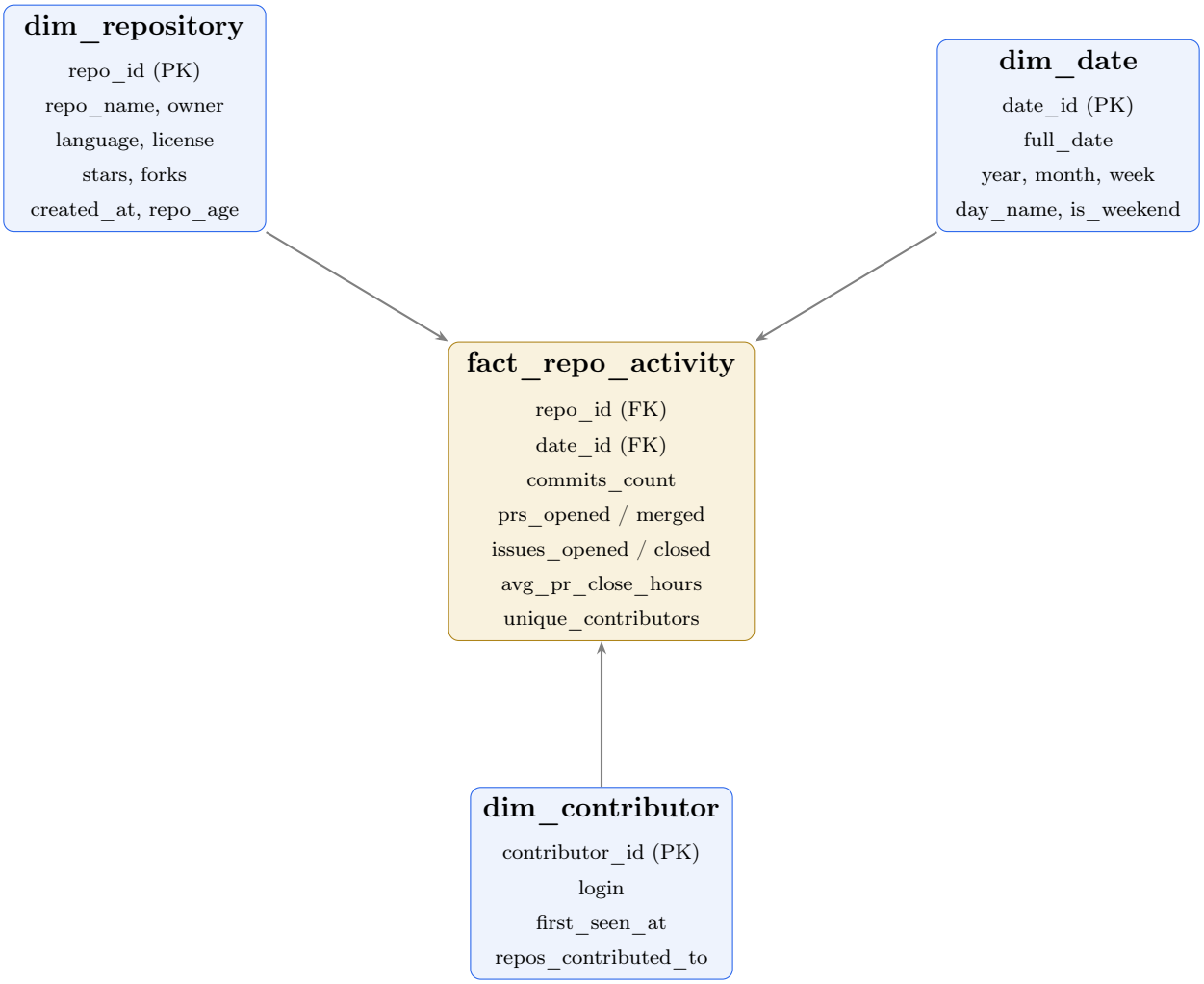
Star Schema Principles

A star schema consists of:

- **Dimension tables**: describe the "who", "what", "where", "when". Examples: `dim_repository`, `dim_date`. Unique primary key.
- **Fact tables**: contain numeric measures (the "how much"). Examples: number of commits, merge duration. Reference dimensions via foreign keys.

The advantage: analytical queries become simple joins between the fact table and the needed dimensions.

7.2 Star Schema Design



7.3 Dimension `dim_repository`

Your turn:

Objective: Create a descriptive dimension for repositories. This table contains **only identity and characteristics** — no aggregated measures.

Expected structure:

- `repo_id`: unique identifier (`owner/repo`).
- `repo_name`, `owner_login`: identity fields.
- `description`, `language`, `license_name`: descriptive attributes.
- `stars_count`, `forks_count`, `watchers_count`: snapshot counters from the API (not aggregated — these are attributes of the repo at extraction time).
- `created_at`, `repo_age_days`: temporal attributes.
- `default_branch`, `has_wiki`, `has_pages`: configuration attributes.

Note: Stars, forks, and watchers are **not** aggregations computed by us — they are snapshot values returned directly by the GitHub API. They describe the repo's state and belong in the dimension.

```
{{ config(materialized='table') }}
```

```
with repo as (
    select * from {{ ref('stg_repositories') }}
)

select
    repo_id,
    repo_name,
    owner_login,
    description,
    language,
    license_name,
    created_at,
    -- TODO: add descriptive attributes only
from repo
```

7.4 Dimension `dim_date`

Your turn:

Objective: Generate a calendar table spanning 5 years.

Expected structure:

- `date_id`: surrogate key in `YYYYMMDD` format (integer).
- `full_date`: the complete date.
- `year`, `month`, `week_of_year`, `day_of_week`: temporal extractions.
- `day_name`, `month_name`: readable names (`Monday`, `January`).
- `is_weekend`: boolean (`dow` = 0 or 6).
- `quarter`: quarter (1 to 4).

Hint: Use DuckDB's `generate_series()` to create the date "spine", then `EXTRACT` and `strftime` to enrich.

```
{{ config(materialized='table') }}

with date_spine as (
    select
        unnest(generate_series(
            current_date - interval '5 years',
            current_date,
            interval '1 day'
        ))::date as full_date
),

enriched as (
    select
        -- TODO
    from date_spine
)

select * from enriched
```

7.5 Dimension `dim_contributor`

Your turn:

Objective: Create a dimension of unique contributors.
Expected structure:

- `contributor_id` / `login`: unique contributor identifier.
- `first_contribution_at`: date of first activity.
- `repos_contributed_to`: number of distinct repos.
- `total_activities`: total number of activities (commits + PRs).

Hint: Combine authors from `stg_commits` and `stg_pull_requests` with a `UNION`, then aggregate by `login`. Exclude the `'unknown'` login.

```
{{ config(materialized='table') }}

-- TODO
```

7.6 Fact Table `fact_repo_activity`

Your turn:

Objective: Create a fact table with daily activity per repo.
Expected structure: For each (repo_id, activity_date) pair, compute:

- `commits_count`, `unique_committers` (from `stg_commits`).
- `prs_opened`, `prs_merged`, `avg_pr_close_hours` (from `stg_pull_requests`).
- `issues_opened`, `issues_closed`, `avg_issue_close_hours` (from `stg_issues`).
- `date_id`: foreign key to `dim_date` in YYYYMMDD format.

Hint: You can create CTEs `daily_commits`, `daily_prs`, `daily_issues` that aggregate by (repo_id, date). Then join them with a `LEFT JOIN` on a `UNION` of all encountered dates.

```
{{ config(materialized='table') }}

 -- TODO
```

Checkpoint Step 3

Run `dbt run -select gold` and verify:

- `dim_repository` contains one row per repo (10 rows) with **descriptive attributes only**.
- `dim_date` covers 5 years of dates (approximately 1826 rows).
- `fact_repo_activity` contains data with consistent measures.
- `dim_contributor` contains unique logins.

8 Step 4 — Scoring Table

Scoring — Business Table

This is the final deliverable for the client. A **composite score** per repository, broken down into thematic sub-scores, enabling objective and actionable comparison.

8.1 Scoring Methodology

The global score (0–100) is the weighted average of **4 thematic sub-scores**. Each sub-score is normalized between 0 and 100 using the **NTILE** method (percentile rank among the 10 repos):

Sub-score	Metrics Used	Weight	Business Logic
Popularity	stars, forks, watchers	20%	Community adoption
Activity	recent commits, contributors	30%	Development dynamism
Responsiveness	PR merge time, issue close time	30%	Maintenance quality
Community	merged PR ratio, closed issue ratio	20%	Community engagement

Why NTILE?

NTILE normalization distributes repos into equal-sized groups based on each metric. This prevents extreme values (one repo with 200,000 stars vs 5,000) from skewing the score. Each metric is transformed into a **relative rank**: what matters is a repo's position compared to others, not the absolute value.

Your turn:

Objective: Create a model `models/gold/scoring_repositories.sql` that produces a ranking of the 10 repos.

Steps to follow:

1. **CTE** `recent_activity`: aggregate metrics from the last 30 days from `fact_repo_activity` (commits, merged PRs, contributors, average close time). Also compute `total_prs`, `merged_prs`, `total_issues`, `closed_issues` over the full history for community ratios.
2. **CTE** `base_metrics`: join `dim_repository` (for descriptive attributes like stars, forks) with `recent_activity` to have all metrics in a single CTE.
3. **CTE** `ranked`: apply `NTILE(10)` on each metric to obtain a relative rank. **Warning**: for reaction times (where "lower = better"), use `ORDER BY ... DESC` in the NTILE.
4. **CTE** `scored`: compute each sub-score by normalizing ranks to 100. Formula: `(sum_of_ranks) * 100.0 / max_possible`.
5. **Final SELECT**: compute `score_global` = weighted average of the 4 sub-scores. Add a `RANK() OVER (ORDER BY score_global DESC)` for ranking.

```
{{ config(materialized='table') }}
```

```
-- TODO
```

8.2 Expected Interpretation

The final table allows directly answering the client's questions:

- **Which repo has the best overall score?** → `ranking` column
- **Which repo is the most recently active?** → `score_activity` column
- **Which repo responds fastest to PRs?** → `score_responsiveness` column
- **Which repos have a strong community but little recent activity?** → compare `score_community` vs `score_activity`

Checkpoint Step 4

Run `dbt run -select scoring_repositories` and verify that all 10 repos appear with scores between 0 and 100 and a ranking from 1 to 10.

9 Step 5 — Tests & Documentation

In a professional dbt project, tests are essential to guarantee pipeline reliability. It is important to distinguish **two complementary test families**:

	A) Data Quality Tests	B) Code / Business Logic Tests
Question asked	"Is the data conformant?"	"Is my SQL logic correct?"
Target	Values in the tables	Transformations and business rules
Examples	No nulls, unique keys, allowed values	Score between 0 and 100, no orphans, correct filtering

9.1 A) Generic Tests (YAML)

dbt's 4 Native Generic Tests

- `not_null` — The column contains no NULL values.
- `unique` — Each column value is unique (primary key).
- `accepted_values` — Values are restricted to a defined list.
- `relationships` — Each value exists in a column of another table (foreign key).

Your turn:

Generic Tests Create two YAML test files:

- `models/silver/_schema.yml`: test the primary keys of the 4 Silver models

(`unique`, `not_null`), and relevant `accepted_values` (e.g.: `pr_state` ∈ {open, closed}, `day_of_week` ∈ {0..6}).

- `models/gold/_schema.yml`: test dimension and fact table keys, `relationships` (referential integrity fact → dim), and scoring keys.

Aim for **at least 15 generic tests** spread across both files.

9.2 B) Singular Tests (SQL)

Singular tests are SQL queries placed in the `tests/` folder. The test **fails** if the query returns rows (each returned row = a violation). Use singular tests for **business logic** that cannot be expressed with generic tests (e.g., cross-column comparisons, multi-table consistency).

Your turn:

Create **at least 3 singular tests** in the `tests/` folder. Here are some ideas:

1. **Ranking consistency**: verify that the repo ranked #1 actually has the highest `score_global`, and no two repos share the same ranking.
2. **Chronological coherence**: verify that `closed_at` is always after `created_at` in `stg_pull_requests` and `stg_issues`.
3. **Merge rate coherence**: verify that `merged_prs` never exceeds `total_prs` in `dim_repository`.
4. **Scoring completeness**: verify that all repositories from `dim_repository` appear in `scoring_repositories` (no repo was lost during scoring).

Reminder: a singular test is a `.sql` file that returns **violations**. Zero rows = test passed.

Example skeleton for a singular test:

```
-- tests/assert_ranking_consistency.sql
-- Fails if the #1 ranked repo does not have the best score
select
    repo_id,
    score_global,
    ranking
from {{ ref('scoring_repositories') }}
where ranking = 1
  and -- TODO: compare against max(score_global)
```

9.3 Execution and Documentation

```
# Run ALL dbt
dbt test

# Generate documentation
dbt docs generate

# Open the docs site in the browser
dbt docs serve
```

The Lineage Graph

The `dbt docs serve` command opens a browser with the **lineage graph**: an interactive graph showing dependencies between your models. You should see the complete chain: sources → stg_* → dim_*/fact_* → scoring.

9.4 Full Pipeline (full refresh mode)

```
# Complete execution in one command
python scripts/load_bronze.py && dbt run && dbt test
```

Checkpoint Step 5

All tests pass. At this point, your full refresh pipeline is complete and validated. Documentation generates without errors and the lineage graph shows the complete chain. You will now move on to the next step: adapting this pipeline for **incremental mode**.

10 Step 6 — Incremental Materialization

From Full Refresh to Incremental

Until now, each `dbt run` execution **recalculates everything** from scratch. In production, this is rarely acceptable: data volumes are too large and compute costs explode. Incremental materialization only processes **new data** that arrived since the last execution.

10.1 Why Go Incremental?

The Full Refresh Problem in Production

Imagine your pipeline runs in production every night:

- After 1 month: 30,000 commits to reprocess every night, of which 29,000 already processed.
- After 1 year: 300,000 commits. The pipeline takes 2 hours instead of 5 minutes.
- Cost: compute time, downtime windows, failure risk on large volumes.

Incremental solves this problem: only process the **delta** (new rows).

10.2 New Data for Testing

Incremental Test Data

To test incremental behavior with real new data, the instructor will provide a **second batch of CSV files** covering the period from **January 1 to February 10, 2026**:

```
data/
  raw/
    # Initial data (Oct-Dec 2025) - already loaded
    raw_repositories_history.csv
```

```
    raw_commits_history.csv
    raw_pull_requests_history.csv
    raw_issues_history.csv
  incremental/
    # New data (Jan 1 - Feb 10, 2026) - provided by instructor
    raw_repositories_YYYY-MM-DD.csv
    raw_commits_YYYY-MM-DD.csv
    raw_pull_requests_YYYY-MM-DD.csv
    raw_issues_YYYY-MM-DD.csv
```

Testing workflow:

1. **Initial state:** your pipeline already works with the Oct–Dec 2025 data (Steps 1–5).
2. **Simulate new arrival:** copy the new CSV files (date by date) from `data/incremental/` into `data/raw/`. This simulates a fresh extraction that includes both old and new data — exactly what would happen in production when the extraction script runs again.
3. **Reload Bronze:** run `python ../scripts/load_bronze.py` to load the updated CSVs into DuckDB.
4. **Run dbt incrementally:** run `dbt run`. Your incremental models should only process the **new rows** (January–February data), not reprocess October–December.

This simulates a realistic production cycle: new data arrives → Bronze is updated → dbt incrementally processes only the delta.

10.3 dbt Incremental Strategies

dbt offers several **incremental strategies**. Your task is to choose the most appropriate one for each model:

Strategy	How it Works	When to Use?
`append`	Inserts new rows without checking for duplicates.	**Immutable** data (logs, events). Fast but no deduplication.
`merge`	Performs a `MERGE` (upsert): updates if the key exists, inserts otherwise.	Data that can be **modified** (statuses, metrics). Requires a `unique_key`.
`delete+insert`	Deletes matching rows then re-inserts.	Alternative to merge when MERGE SQL is not supported.

How does `is_incremental()` work?

On the **first** execution, dbt creates the complete table. On **subsequent** executions, the `{% if is_incremental() %}` block is activated and filters for new rows. The `{{ this }}` function references the already materialized table.

```
-- General pattern
{% if is_incremental() %}
```

```
  where committed_at > (select max(committed_at) from {{ this }})
{% endif %}
```

Your turn:

Objective: Adapt **at least 2 Silver models** to work in incremental mode. For each model:

1. Choose the most appropriate **strategy** (`append`, `merge`, or `delete+insert`) and justify your choice.
2. Identify the **incremental column** (the date used to determine "new" rows).
3. Identify the **unique_key** (if you choose `merge`).
4. Modify the `config()` and add the `is_incremental()` block.

Reflection questions:

- Can a commit be modified after the fact? What impact does this have on strategy choice?
- An issue can change state (open → closed). How do you handle this case?
- Should the Gold layer also be made incremental? What problems does this cause?

Here is a skeleton to help you:

```
-- Example: stg_commits.sql as incremental
{{
  config(
    materialized='incremental',
    schema='silver',
    unique_key='...',           -- TODO: Which key?
    incremental_strategy='...' -- TODO: Which strategy?
  )
}}

with source as (
    select * from {{ source('bronze', 'raw_commits') }}
),

cleaned as (
    -- ... (same logic as your full refresh version)
)

select * from cleaned

-- TODO: Add the incremental filter
{% if is_incremental() %}
  where ... > (select max(...) from {{ this }})
{% endif %}
```

10.4 Testing Incremental

```
# --- Step A: Verify your initial pipeline (Oct-Dec 2025) ---
dbt run
dbt show --inline "select count(*) as nb from silver.stg_commits"
# Note the count (e.g., 5000 rows)
```

```
# --- Step B: Load the new data (Jan-Feb 2026) ---
# Copy incremental CSVs into data/raw/
cp ../data/incremental/raw_commits_2026-01-01.csv ../data/raw/

# Reload Bronze with new + old data
python ../scripts/load_bronze.py

# --- Step C: Run dbt incrementally ---
dbt run --select silver
# Observe the logs: only new rows should be processed

dbt show --inline "select count(*) as nb from silver.stg_commits"
# Count should now be higher

# --- Step D: Verify idempotency ---
dbt run --select silver
# Second run: 0 new rows processed (nothing changed in Bronze)

# --- Step E: Full refresh if needed ---
dbt run --select silver --full-refresh
# Rebuilds everything from scratch
```

What to observe in the logs

When running `dbt run` after loading new data, pay attention to the log output:

- **Incremental models** should show a row count matching only the **new** data (Jan–Feb 2026), not the full history.
- **Non-incremental models** (views, tables in Gold) will still reprocess everything.
- Compare the row counts **before and after** to confirm the delta was correctly captured.

Checkpoint Step 6

Verify that:

- After loading new data and running `dbt run`, only **new rows** (Jan–Feb 2026) are processed by incremental models.
- A second `dbt run` (with no new data) processes **0 new rows**.
- `dbt run -full-refresh` correctly rebuilds the complete table from scratch.
- All `dbt test` still pass after the incremental load.

11 Step 7 — dbt Snapshots

Snapshots — Capturing Change History

dbt snapshots capture the evolution of data over time (SCD Type 2). Each time a value changes, dbt creates a new version of the row with `dbt_valid_from` and `dbt_valid_to` timestamps.

11.1 Why Snapshot?

Some data changes frequently without native history. For example, a repo's `stars` count increases daily, but the API only returns the *current* value. If you want to track the evolution of stars over time, you need to **snapshot** the table regularly.

The snapshot automatically creates a history:

repo_id	stars	dbt_valid_from	dbt_valid_to
react	231,000	2026-01-01	2026-01-15
react	232,500	2026-01-15	*null* (active)

11.2 Choosing the Table to Snapshot

The `raw_repositories` table is the best candidate: its metrics (`stargazers_count`, `forks_count`, `open_issues_count`) change daily, and the API only provides the current value.

11.3 Strategy: `check` vs `timestamp`

	timestamp	check
Detection	Compares an `updated_at` column	Compares values of specific columns
Advantage	Fast, single column	No need for a reliable timestamp column
Limitation	Requires a reliable `updated_at`	Slower (compares N columns)

Your turn:

Create a file `snapshots/snap_repositories.yml`.

Instructions:

- Source: the `raw_repositories` table (directly from Bronze).
- `unique_key`: the field that identifies each repo.
- `strategy`: choose between `check` and `timestamp` based on the available data.
- If `check`: list the columns to monitor (`check_cols`).
- If `timestamp`: specify the temporal column (`updated_at`).

```
# snapshots/snap_repositories.yml
snapshots:
  - name: snap_repositories
    relation: source('bronze', 'raw_repositories')
    config:
      schema: snapshots
      unique_key: "..."
      strategy: "..."
      # TODO: add check_cols or updated_at depending on strategy
      # check_cols:
      #   - col1
      #   - col2
```

```
      # updated_at: "..."
```

11.4 Running Snapshots

```
# First execution: creates the snapshot table
dbt snapshot

# Verify the contents
dbt show --inline "
  select full_name, stargazers_count,
         dbt_valid_from, dbt_valid_to
  from snapshots.snap_repositories
  order by full_name, dbt_valid_from
"
```

Checkpoint Step 7

Verify that the `snap_repositories` table contains the `dbt_valid_from` and `dbt_valid_to` columns. After the first execution, all rows have `dbt_valid_to = NULL` (active version). If you load updated data and re-run `dbt snapshot`, old rows will be "closed" and new versions created.

12 Expected Deliverables

Your final dbt project must contain the following structure:

```
github_analytics/
  dbt_project.yml
  models/
    bronze/
      _sources.yml
    silver/
      stg_repositories.sql
      stg_commits.sql
      stg_pull_requests.sql
      stg_issues.sql
      _schema.yml
    gold/
      dim_repository.sql
      dim_date.sql
      dim_contributor.sql
      fact_repo_activity.sql
      scoring_repositories.sql
      _schema.yml
  tests/
    assert_ranking_consistency.sql
    assert_chronological_coherence.sql
    assert_merge_rate_coherence.sql
    assert_scoring_completeness.sql
  snapshots/
    snap_repositories.yml
```

The complete pipeline must execute without errors:

```
python scripts/load_bronze.py   # Load CSVs into DuckDB
dbt run                          # Build all models
dbt test                         # Run all tests
dbt docs generate                # Generate documentation
```

13 References and Resources

- **dbt Documentation**: https://docs.getdbt.com/
- **DuckDB Documentation**: https://duckdb.org/docs/
- **dbt-duckdb adapter**: https://github.com/duckdb/dbt-duckdb
- **GitHub REST API v3**: https://docs.github.com/en/rest
- **Medallion Architecture**: https://www.databricks.com/glossary/medallion-architecture

Appendix A — Essential dbt Commands

Command	Usage
`dbt debug`	Verify connection and configuration
`dbt run`	Run all models
`dbt run -select silver`	Run only Silver models
`dbt run -select +scoring_repositories`	Run a model and its dependencies
`dbt run -full-refresh`	Force complete recreation
`dbt test`	Run all tests
`dbt test -select scoring_repositories`	Test a specific model
`dbt snapshot`	Run snapshots
`dbt docs generate`	Generate documentation
`dbt docs serve`	Open documentation in the browser
`dbt show -inline "SELECT ..."`	Run an ad hoc query

Appendix B — Glossary

Bronze / Silver / Gold
: The three layers of the Medallion architecture, corresponding respectively to raw, cleaned, and business-ready data.

CTE (Common Table Expression)
: A `WITH ... AS ()` block in SQL, allowing complex queries to be broken down into named steps.

ELT (Extract-Load-Transform)
An architecture where data is first loaded raw then transformed within the warehouse.

Lineage
The dependency graph between dbt models, generated automatically.

Materialization
How dbt persists a model: `view`, `table`, `incremental`, or `ephemeral`.

NTILE
An analytical SQL function that distributes rows into n equal-sized groups, used here to normalize scores.

SCD Type 2 (Slowly Changing Dimension)
A history management technique where each change creates a new version of the row with validity timestamps.

Snapshot
A dbt mechanism to automatically implement SCD Type 2.

Star Schema
A dimensional model composed of fact tables (measures) and dimension tables (context), optimized for analytical queries.